1. Project Title

Hospital Appointment System

2. Student Details

Enrollment Number: IU2541230411
Project Code: CPP02
GitHub Repository Link:

https://github.com/chauhandivyang369-oss/IU2541230411_PROJECT.git

3. Problem Statement

In many small hospitals and clinics, patient details, doctor details and appointments are still written in registers. This causes several real-world problems: records get lost or damaged, searching for one patient takes a long time, the same doctor is accidentally given two patients at the same time (double booking), and it is difficult to know how many appointments were booked, cancelled or completed in a day.

This project solves these problems with a console based Hospital Appointment System written in C++. The system stores patient, doctor and appointment records, allows the receptionist to add, view, search, update and delete records, books appointments while preventing double booking, creates a summary report, and saves everything in text files so that the data is still available when the program is closed and opened again.

4. Objectives

- To build a menu driven C++ program that manages patients, doctors and appointments.
- To apply Object Oriented Programming concepts: classes and objects, encapsulation, constructors, destructors, inheritance and runtime polymorphism.
- To use dynamic memory (new[] and delete[]) for storing records and release it correctly.
- To store data permanently using file handling so that records are available after restarting the program.
- To validate every user input and handle errors safely using exception handling.
- To keep the code modular by using separate .h and .cpp files, with main.cpp only showing the menu.
- To prepare UML diagrams (Use Case, Class and two Sequence diagrams) and document the project properly.

5. Features

The program shows a main menu with nine options. All options are handled by the Hospital class.

No.	Menu Option	What it does
1	Add Record	Adds a new patient (name, age, phone, disease) or a new doctor (name, age, phone, specialization, fee). IDs are generated automatically (patients 1001+, doctors 2001+).
2	Display Records	Shows all patients, all doctors, all appointments, or all people together (patients + doctors) using one Person pointer.
3	Search Record	Search a patient by ID, a doctor by ID, doctors by specialization, or an appointment by ID.
4	Update Record	Update a patient's phone and disease, a doctor's fee, or reschedule a booked appointment.
5	Delete Record	Delete a patient, doctor or appointment. A patient or doctor with a booked appointment cannot be deleted.
6	Book / Manage Appointment (Main Process)	Book an appointment (patient + doctor + date + time slot) with double booking check, cancel a booked appointment, or mark it completed.
7	Report	Shows totals of patients, doctors and appointments, booked/completed/cancelled counts, revenue from completed appointments and appointments per doctor.
8	Save / Load	Saves all records to text files or loads them again from the files.
9	Exit	Saves the data automatically, releases the dynamic memory and closes the program.

Other features: automatic loading of old records at start-up, input validation (menu range, names, phone number, date, empty text), and clear error messages through exception handling.

Project Folder Structure

Folder / File	Contents
Source/	main.cpp, Person.h/.cpp, Patient.h/.cpp, Doctor.h/.cpp, Appointment.h/.cpp, Hospital.h/.cpp, FileManager.h/.cpp, Input.h/.cpp
Data/	patients.txt, doctors.txt, appointments.txt
Diagrams/	UseCase.png, ClassDiagram.png, Sequence1.png, Sequence2.png
Executable/	Project (compiled program)
Screenshots/	Program screenshots (12 images)
README.txt	How to compile and run the project
ProjectReport.pdf	This report

6. Classes and Responsibilities

The project has the six classes required by the blueprint. Person is the base class, Patient and Doctor are derived classes, Appointment stores one booking, Hospital is the system class that

does the work and FileManager handles the files. A small helper file Input.h/Input.cpp contains simple functions for validated keyboard input.

Class	Data members (all private)	Important member functions	Responsibility
Person (base class)	name : string age : int phone : string	Person(), Person(n,a,p), ~Person(), getName(), setPhone(), display(), toFileString()	Holds details common to every person. display() and toFileString() are virtual.
Patient (derived from Person)	id : int disease : string	Patient(), Patient(i,n,a,p,d), ~Patient(), getId(), setDisease(), display(), toFileString()	Stores one patient. Overrides display() and toFileString().
Doctor (derived from Person)	id : int specialization : string fee : int	Doctor(), Doctor(i,n,a,p,s,f), ~Doctor(), getId(), getSpecialization(), getFee(), setFee(), display(), toFileString()	Stores one doctor and the consultation fee. Overrides display() and toFileString().
Appointment	id, patientId, doctorId : int date, slot, status : string	Appointment(), Appointment(...), getters, setStatus(), setSchedule(), clashesWith(), display(), toFileString()	Stores one booking. clashesWith() checks whether the same doctor is already booked at the same date and slot.
Hospital (system class)	patients*, doctors*, appointments* (dynamic arrays) patientCount, doctorCount, appointmentCount, capacity nextPatientId, nextDoctorId, nextAppointmentId fileManager : FileManager	Hospital(), ~Hospital(), addRecord(), displayRecords(), searchRecord(), updateRecord(), deleteRecord(), appointmentProcess(), report(), saveData(), loadData(), findIndex(), showAppointment()	Owns all records and contains the business logic of every menu option. The destructor releases the dynamic memory.
FileManager	patientFile, doctorFile, appointmentFile : string	FileManager(), saveAll(), loadAll(), splitLine()	Writes all records to the text files and reads them back. It is a friend class of Hospital.

Input.cpp (helper functions, not a class): readInt() reads a number in a range, readText() reads non-empty text, readName() accepts only letters, spaces and '.', readPhone() accepts exactly 10 digits and readDate() accepts a real date in DD-MM-YYYY format.

7. OOP Concepts Used

7.1 Classes and Objects

A class is a blueprint and an object is a real copy of it. In main.cpp the line Hospital hospital; creates the system object. Inside Hospital, objects of Patient, Doctor and Appointment are created, for example: patients[patientCount] = Patient(nextPatientId, name, age, phone, disease);

7.2 Encapsulation

All data members are private. They can be used only through public member functions such as getId(), getFee(), setFee() and setPhone(). Because of this, other code cannot put wrong values directly into an object.

```
class Person {
private:
   string name;  int age;  string phone;
public:
   string getName() const;
   void setPhone(string p);
};
```

7.3 Constructors (default and parameterized) and Destructors

Every entity class has a default constructor and a parameterized constructor. The default constructor is really needed, because new Patient[capacity] creates many objects without giving values. The parameterized constructor of Person also checks the name and age and throws an exception if they are wrong.

```
Person::Person() { name = "Unknown"; age = 0; phone = "0000000000"; }

Person::Person(string n, int a, string p) {
   if (n.empty()) throw invalid_argument("Name cannot be empty");
   if (a < 0 || a > 120) throw invalid_argument("Age must be between 0 and 120");
   name = n;  age = a;  phone = p;
}
```

Person, Patient and Doctor have destructors (Person's destructor is virtual). The Hospital destructor is the important one: it gives the dynamic memory back and prints "Memory released successfully (delete[] used)." when the program ends.

7.4 Inheritance

Patient and Doctor are both a kind of Person, so they inherit name, age, phone and the common functions from Person instead of repeating them. The derived constructor passes the common values to the base constructor.

```
class Patient : public Person { ... };

Patient::Patient(int i, string n, int a, string p, string d) : Person(n, a, p) {
   id = i;  disease = d;
}
```

7.5 Runtime Polymorphism

display() and toFileString() are declared virtual in Person and overridden in Patient and Doctor. In Display Records option 4, one Person pointer is used for both patients and doctors, and the correct version of display() is chosen automatically while the program is running.

```
Person *p;
for (int i = 0; i < patientCount; i++) { p = &patients[i]; p->display(); }  // Patient::display()
for (int i = 0; i < doctorCount; i++)  { p = &doctors[i];  p->display(); }  // Doctor::display()
```

7.6 Dynamic Memory

The Hospital constructor creates three arrays on the heap with new[]. The destructor releases them with delete[].

```
capacity = 50;
patients     = new Patient[capacity];
doctors      = new Doctor[capacity];
appointments = new Appointment[capacity];

Hospital::~Hospital() {
   delete[] patients;  delete[] doctors;  delete[] appointments;
}
```

Purpose: the arrays are created while the program runs, so the memory belongs to the Hospital object and exists as long as the system runs. Using delete[] in the destructor makes sure the memory is returned to the system and there is no memory leak. The capacity (50 records of each type) is stored in one variable, so it can be changed easily.

7.7 Validation and Exception Handling

Every value typed by the user is checked by the functions in Input.cpp (range of numbers, only letters in names, 10 digit phone number, valid calendar date, no empty text and no '|' character). Logical errors such as "Patient not found" or "Doctor already booked" are reported by throwing runtime_error from the Hospital functions. main.cpp catches every exception in one try/catch block and prints the message, so the program never crashes.

```
try {
   // switch(choice) calls hospital.addRecord(), hospital.searchRecord() ...
}
catch (exception &e) {
   cout << "Error: " << e.what() << endl;
}
```

7.8 Modular Programming and Friend Class

Every class has its own .h (declaration) and .cpp (definition) file. main.cpp only creates the Hospital object, shows the menu and calls Hospital functions. Hospital declares FileManager as a friend class, so FileManager can directly read and fill the Hospital arrays while saving and loading, without adding many extra getter functions.

8. File Handling

Data stored: all patients, all doctors and all appointments. Three plain text files are used, one record per line, fields separated by the '|' character.

File	Format of one line	Example
Data/patients.txt	id|name|age|phone|disease	1001|Rahul Sharma|34|9876543210|Fever
Data/doctors.txt	id|name|age|phone|specialization|fee	2001|Dr. Kiran Mehta|45|9811100011|General Physician|600
Data/appointments.txt	id|patientId|doctorId|date|slot|status	3001|1001|2001|14-10-2026|10:00 AM|Completed

Saving process

When the user selects Save (or exits), Hospital::saveData() calls FileManager::saveAll(). It opens the three files with ofstream and writes the result of toFileString() of every record, one line each. If a file cannot be opened, a runtime_error is thrown and shown to the user.

Loading process

At program start Hospital::loadData() calls FileManager::loadAll(). It reads each file line by line with getline(), cuts the line at '|' using splitLine(), converts numbers with stoi() and creates the objects again. A damaged line is skipped with a message. After loading, the next ID counters continue after the highest ID found, so IDs are never repeated.

Persistence testing

Records were added, the program was closed (automatic save), and then the program was started again. The start-up message "Data loaded: 5 patients, 3 doctors, 5 appointments." and the display of all records (Screenshot 1) show that the data was restored correctly.

9. UML Diagrams

9.1 Use Case Diagram

Three actors use the system: the Receptionist (Admin) who operates every function, the Patient who searches records and books or cancels appointments through the receptionist, and the Doctor who views records and completes appointments. The diagram has ten use cases.

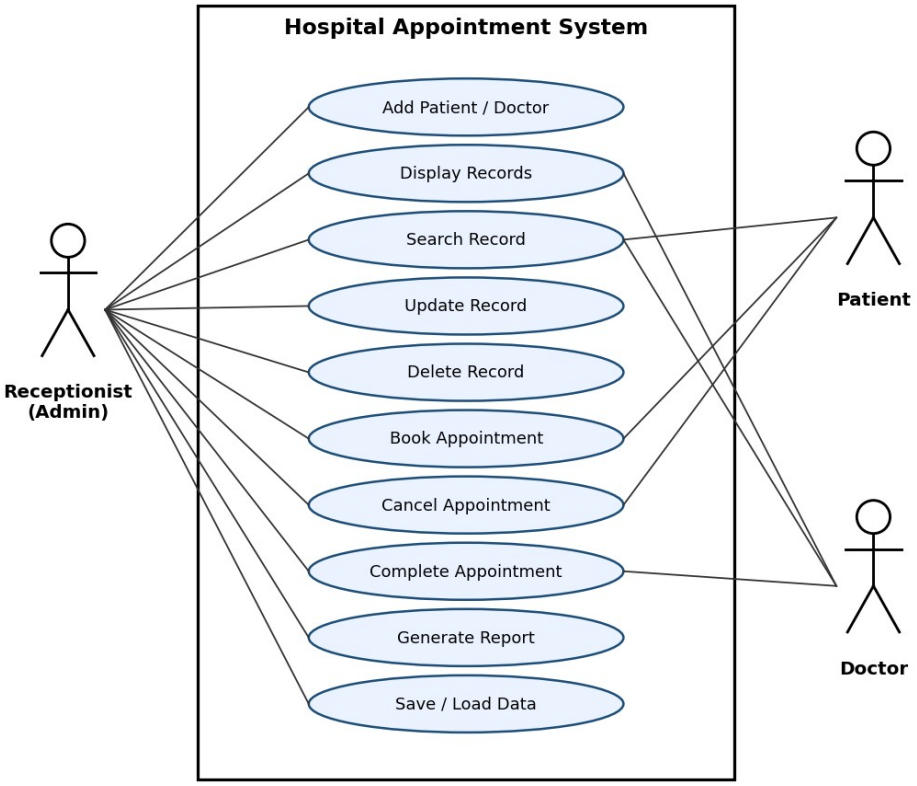


Figure 1: Use Case Diagram

9.2 Class Diagram

The class diagram shows data members, member functions, visibility (- private, + public) and relationships: inheritance (Patient and Doctor from Person), composition (Hospital owns the record arrays and the FileManager), friend access (FileManager and Hospital) and association (Appointment refers to Patient and Doctor by ID).

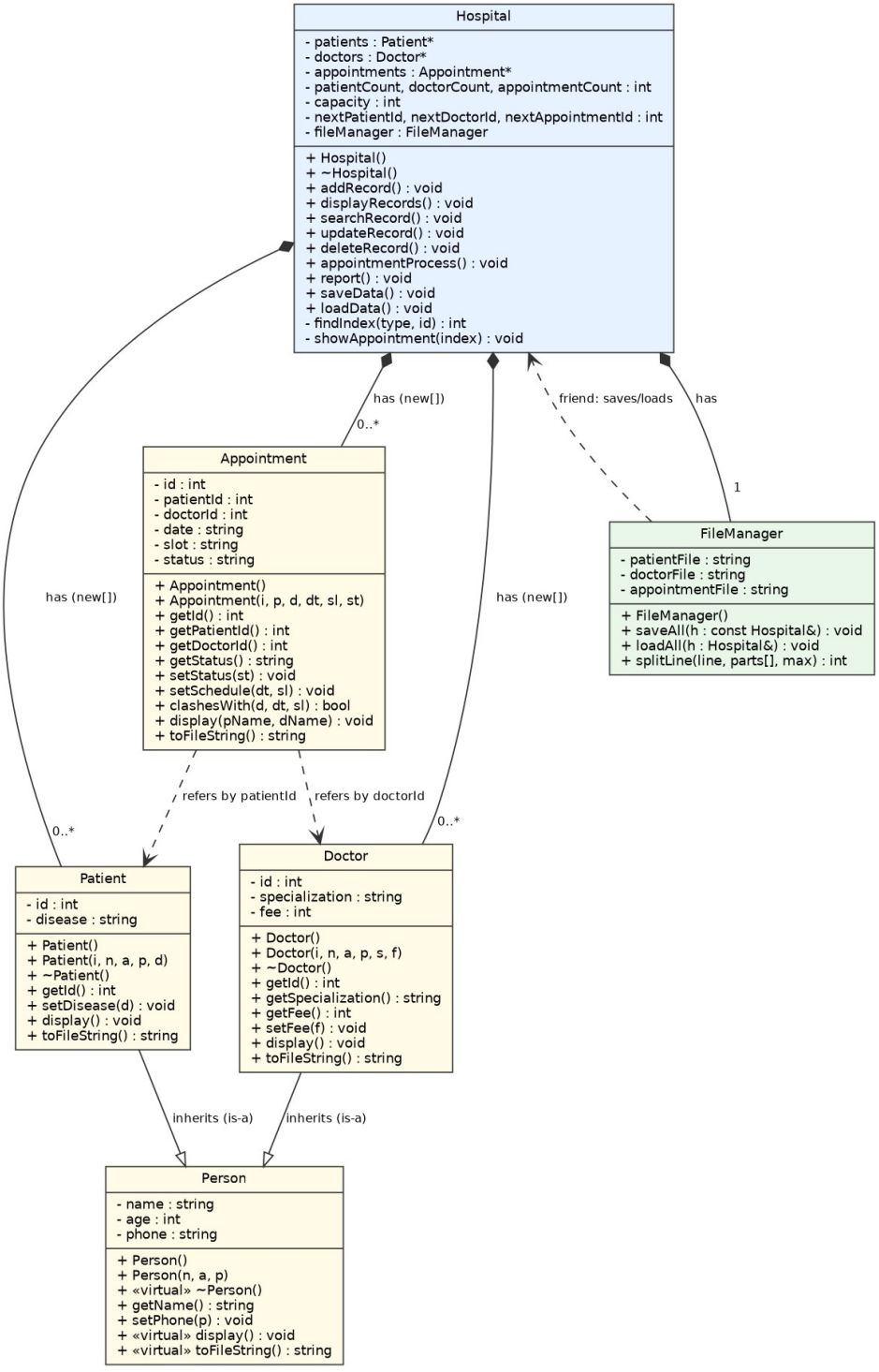


Figure 2: Class Diagram

9.3 Sequence Diagram 1 - Book Appointment (main transaction)

The receptionist chooses option 6. main.cpp calls Hospital::appointmentProcess(). Hospital reads validated input, checks that the patient and doctor exist, calls clashesWith() on appointments to prevent double booking, creates the new Appointment object and shows the fee read from the Doctor.

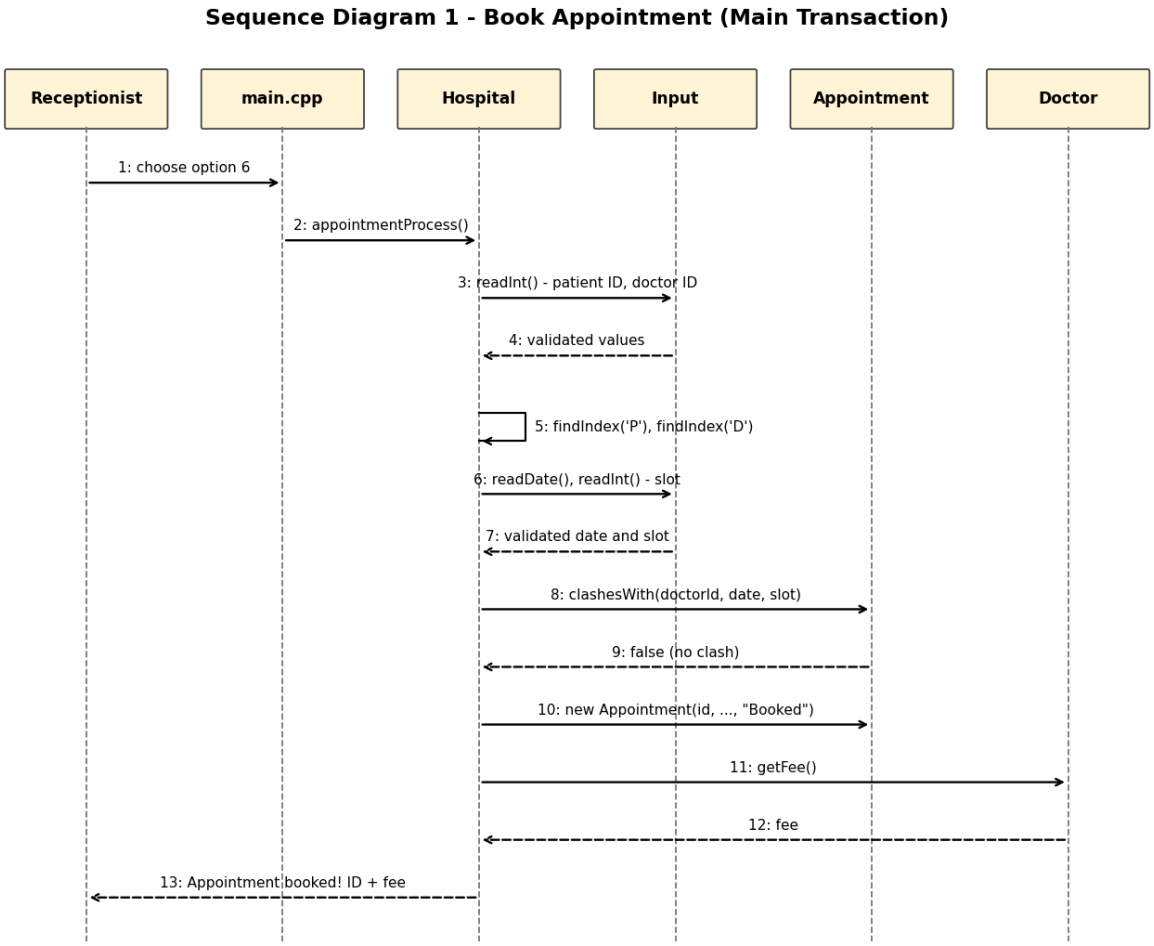


Figure 3: Sequence Diagram 1 - Book Appointment

9.4 Sequence Diagram 2 - Save Data to Files

Option 8 calls Hospital::saveData(), which calls FileManager::saveAll(). FileManager opens the data files, calls the virtual toFileString() of each record and writes the lines to the files.

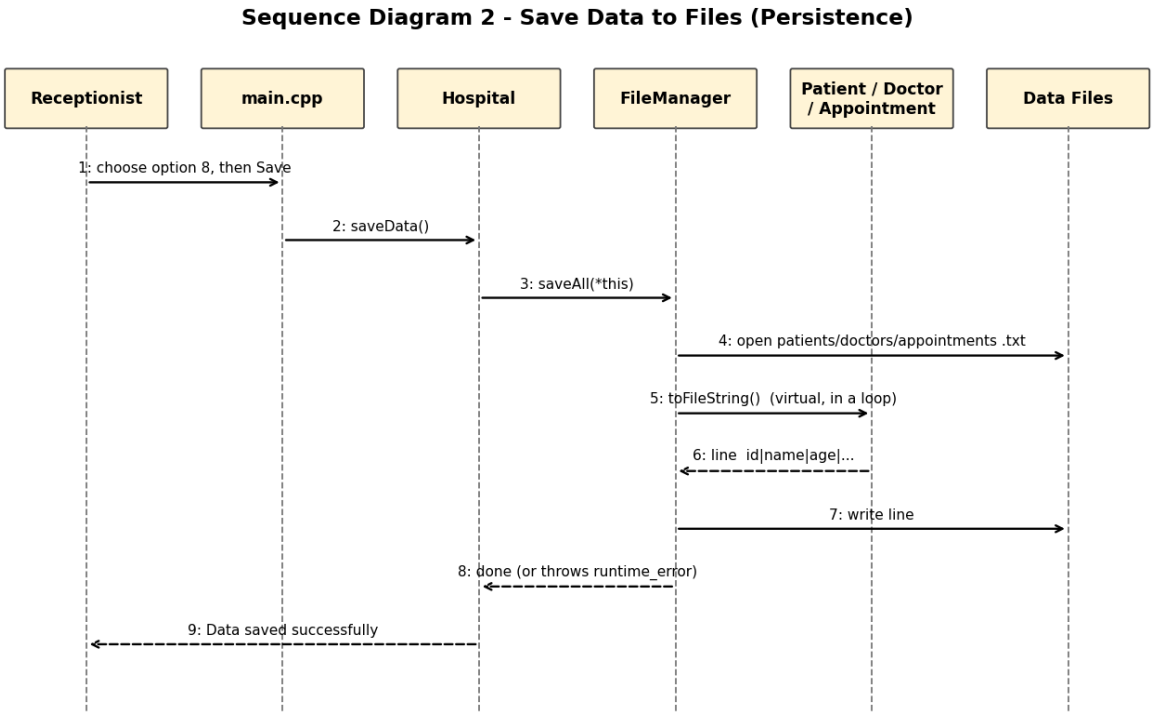


Figure 4: Sequence Diagram 2 - Save Data

10. Program Workflow

1. The program starts and main() creates the Hospital object. The constructor allocates the dynamic arrays with new[].
2. main() calls hospital.loadData(). FileManager reads the three data files and the old records are loaded. If the files are empty or missing, the system starts empty.
3. The main menu with 9 options is displayed inside a do-while loop.
4. The user enters a choice. The number is validated by readInt() (1 to 9).
5. A switch statement in main.cpp calls the matching Hospital function (addRecord, displayRecords, searchRecord, updateRecord, deleteRecord, appointmentProcess, report or save/load).
6. The Hospital function shows a sub-menu if needed, reads and validates the input, works on the entity objects (Patient, Doctor, Appointment) and prints the result.
7. If anything goes wrong, a runtime_error is thrown. main.cpp catches it, prints "Error: ..." and the menu is shown again.
8. The loop continues until the user selects 9 (Exit).
9. On exit, hospital.saveData() writes all records to the files through FileManager.
10. When main() ends, the Hospital destructor runs, delete[] releases the memory and the program closes.

11. Screenshots

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> g++ *.cpp -o ../Executable/Project.exe
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable
>> .\Project.exe
Data loaded: 5 patients, 3 doctors, 5 appointments.

======================================
     HOSPITAL APPOINTMENT SYSTEM
======================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9):
```

Screenshot 1: Main Menu

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable

======================================
     HOSPITAL APPOINTMENT SYSTEM
======================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 1

--- ADD RECORD ---
1. Patient
2. Doctor
Enter choice: 1
Patient name: Rahul
Age (1-120): 30
Phone (10 digits): 9655654785
Disease / problem: fever
Patient added successfully. Patient ID = 1006
```

Screenshot 2: 1. Add Record (1)

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable

======================================
     HOSPITAL APPOINTMENT SYSTEM
======================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 1

--- ADD RECORD ---
1. Patient
2. Doctor
Enter choice: 2
Doctor name: shyam
Age (25-80): 25
Phone (10 digits): 9855554463
Specialization: MBBS
Consultation fee (100-5000): 2000
Doctor added successfully. Doctor ID = 2004
```

Screenshot 3: 1. Add Record (2)

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable

======================================
     HOSPITAL APPOINTMENT SYSTEM
======================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 2

--- DISPLAY RECORDS ---
1. Patients
2. Doctors
3. Appointments
4. All People (Patients + Doctors)
Enter choice: 1

[Patient] ID: 1001 | Name: Rahul Sharma | Age: 34 | Phone: 9876543210 | Disease: Fever
[Patient] ID: 1002 | Name: Priya Patel | Age: 28 | Phone: 9825099999 | Disease: Skin Allergy (Chronic)
[Patient] ID: 1003 | Name: Amit Shah | Age: 45 | Phone: 9898989898 | Disease: Diabetes
[Patient] ID: 1004 | Name: Neha Desai | Age: 52 | Phone: 9712345678 | Disease: Back Pain
[Patient] ID: 1005 | Name: Rohan Verma | Age: 25 | Phone: 9123456789 | Disease: Headache
[Patient] ID: 1006 | Name: Rahul  | Age: 30 | Phone: 9655654785 | Disease: fever
```

Screenshot 4: 2. Display Records (1)

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable

======================================
     HOSPITAL APPOINTMENT SYSTEM
======================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 2

--- DISPLAY RECORDS ---
1. Patients
2. Doctors
3. Appointments
4. All People (Patients + Doctors)
Enter choice: 2

[Doctor]  ID: 2001 | Name: Dr. Kiran Mehta | Age: 45 | Phone: 9811100011 | Speciality: General Physician | Fee: Rs.600
[Doctor]  ID: 2002 | Name: Dr. Anita Joshi | Age: 39 | Phone: 9811122233 | Speciality: Dermatologist | Fee: Rs.800
[Doctor]  ID: 2003 | Name: Dr. Suresh Rao | Age: 50 | Phone: 9822233344 | Speciality: Orthopedic | Fee: Rs.1000
[Doctor]  ID: 2004 | Name: shyam | Age: 25 | Phone: 9855554463 | Speciality: MBBS | Fee: Rs.2000
```

Screenshot 5: 2. Display Records (2)

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable

======================================
     HOSPITAL APPOINTMENT SYSTEM
======================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 2

--- DISPLAY RECORDS ---
1. Patients
2. Doctors
3. Appointments
4. All People (Patients + Doctors)
Enter choice: 3

[Appointment] ID: 3001 | Patient: Rahul Sharma (1001) | Doctor: Dr. Kiran Mehta (2001) | 14-10-2026 10:00 AM | Status: Completed
[Appointment] ID: 3002 | Patient: Priya Patel (1002) | Doctor: Dr. Anita Joshi (2002) | 14-10-2026 11:00 AM | Status: Booked
[Appointment] ID: 3003 | Patient: Amit Shah (1003) | Doctor: Dr. Suresh Rao (2003) | 17-10-2026 11:00 AM | Status: Booked
[Appointment] ID: 3005 | Patient: Rahul Sharma (1001) | Doctor: Dr. Anita Joshi (2002) | 16-10-2026 03:00 PM | Status: Booked
[Appointment] ID: 3006 | Patient: Priya Patel (1002) | Doctor: Dr. Anita Joshi (2002) | 15-10-2026 04:00 PM | Status: Booked
```

Screenshot 6: 2. Display Records (3)

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable

====================================
     HOSPITAL APPOINTMENT SYSTEM
====================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 2

--- DISPLAY RECORDS ---
1. Patients
2. Doctors
3. Appointments
4. All People (Patients + Doctors)
Enter choice: 4

[Patient] ID: 1001 | Name: Rahul Sharma | Age: 34 | Phone: 9876543210 | Disease: Fever
[Patient] ID: 1002 | Name: Priya Patel | Age: 28 | Phone: 9825099999 | Disease: Skin Allergy (Chronic)
[Patient] ID: 1003 | Name: Amit Shah | Age: 45 | Phone: 9898989898 | Disease: Diabetes
[Patient] ID: 1004 | Name: Neha Desai | Age: 52 | Phone: 9712345678 | Disease: Back Pain
[Patient] ID: 1005 | Name: Rohan Verma | Age: 25 | Phone: 9123456789 | Disease: Headache
[Patient] ID: 1006 | Name: Rahul  | Age: 30 | Phone: 9655654785 | Disease: fever
[Doctor]  ID: 2001 | Name: Dr. Kiran Mehta | Age: 45 | Phone: 9811100011 | Speciality: General Physician | Fee: Rs.600
[Doctor]  ID: 2002 | Name: Dr. Anita Joshi | Age: 39 | Phone: 9811122233 | Speciality: Dermatologist | Fee: Rs.800
[Doctor]  ID: 2003 | Name: Dr. Suresh Rao | Age: 50 | Phone: 9822233344 | Speciality: Orthopedic | Fee: Rs.1000
[Doctor]  ID: 2004 | Name: shyam | Age: 25 | Phone: 9855554463 | Speciality: MBBS | Fee: Rs.2000
```

Screenshot 7: 2. Display Records (4)

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable

====================================
     HOSPITAL APPOINTMENT SYSTEM
====================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 3

--- SEARCH RECORD ---
1. Patient by ID
2. Doctor by ID
3. Doctors by specialization
4. Appointment by ID
Enter choice: 1
Enter ID: 1001
[Patient] ID: 1001 | Name: Rahul Sharma | Age: 34 | Phone: 9876543210 | Disease: Fever
```

Screenshot 8: 3. Search Record (1)

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable
====================================
     HOSPITAL APPOINTMENT SYSTEM
====================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 3

--- SEARCH RECORD ---
1. Patient by ID
2. Doctor by ID
3. Doctors by specialization
4. Appointment by ID
Enter choice: 2
Enter ID: 2001
[Doctor]  ID: 2001 | Name: Dr. Kiran Mehta | Age: 45 | Phone: 9811100011 | Speciality: General Physician | Fee: Rs.600
```

Screenshot 9: 3. Search Record (2)

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable
=====================================
     HOSPITAL APPOINTMENT SYSTEM
=====================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 3

--- SEARCH RECORD ---
1. Patient by ID
2. Doctor by ID
3. Doctors by specialization
4. Appointment by ID
Enter choice: 3
Enter specialization:  MBBS
No doctor found with this specialization.
```

Screenshot 10: 3. Search Record (3)

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable

=====================================
     HOSPITAL APPOINTMENT SYSTEM
=====================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 3

--- SEARCH RECORD ---
1. Patient by ID
2. Doctor by ID
3. Doctors by specialization
4. Appointment by ID
Enter choice: 4
Enter ID: 1001

Error: Appointment not found.
```

Screenshot 11: 3. Search Record (4)

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable

======================================
     HOSPITAL APPOINTMENT SYSTEM
======================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 4

--- UPDATE RECORD ---
1. Patient (phone and disease)
2. Doctor (fee)
3. Reschedule appointment
Enter choice: 1
Enter ID: 1001
New phone (10 digits): 9622233441
New disease / problem: fever
Patient updated successfully.
```

Screenshot 12: 4. Update Record (1)

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable

======================================
     HOSPITAL APPOINTMENT SYSTEM
======================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 4

--- UPDATE RECORD ---
1. Patient (phone and disease)
2. Doctor (fee)
3. Reschedule appointment
Enter choice: 2
Enter ID: 2001
New fee (100-5000): 3000
Doctor fee updated successfully.
```

Screenshot 13: 4. Update Record (2)

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable

======================================
     HOSPITAL APPOINTMENT SYSTEM
======================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 4

--- UPDATE RECORD ---
1. Patient (phone and disease)
2. Doctor (fee)
3. Reschedule appointment
Enter choice: 3
Enter ID: 3001

Error: Only a Booked appointment can be rescheduled.
```

Screenshot 14: 4. Update Record (3)

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable

======================================
     HOSPITAL APPOINTMENT SYSTEM
======================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 5

--- DELETE RECORD ---
1. Patient
2. Doctor
3. Appointment
Enter choice: 1
Enter ID to delete: 1001

Error: Patient has a booked appointment. Cancel it first.
```

Screenshot 15: 5. Delete Record (1)

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable
======================================
     HOSPITAL APPOINTMENT SYSTEM
======================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 5

--- DELETE RECORD ---
1. Patient
2. Doctor
3. Appointment
Enter choice: 2
Enter ID to delete: 3001

Error: Doctor not found.
```

Screenshot 16: 5. Delete Record (2)

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable

======================================
     HOSPITAL APPOINTMENT SYSTEM
======================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 5

--- DELETE RECORD ---
1. Patient
2. Doctor
3. Appointment
Enter choice: 3
Enter ID to delete: 3001
Appointment deleted successfully.
```

Screenshot 17: 5. Delete Record (3)

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable

======================================
     HOSPITAL APPOINTMENT SYSTEM
======================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 6

--- APPOINTMENT PROCESS ---
1. Book Appointment
2. Cancel Appointment
3. Mark Appointment Completed
Enter choice: 1
Patient ID: 1001
Doctor ID: 2002
Appointment date (DD-MM-YYYY): 04-02-2026
1. 10:00 AM   2. 11:00 AM   3. 12:00 PM   4. 02:00 PM   5. 03:00 PM   6. 04:00 PM
Choose slot (1-6): 2
Appointment booked! Appointment ID = 3007 | Fee to pay: Rs.800
```

Screenshot 18: 6. Book / Manage Appointment (1)

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable

======================================
     HOSPITAL APPOINTMENT SYSTEM
======================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 6

--- APPOINTMENT PROCESS ---
1. Book Appointment
2. Cancel Appointment
3. Mark Appointment Completed
Enter choice: 2
Appointment ID: 3007
Appointment cancelled.
```

Screenshot 19: 6. Book / Manage Appointment (2)

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable

======================================
     HOSPITAL APPOINTMENT SYSTEM
======================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 6

--- APPOINTMENT PROCESS ---
1. Book Appointment
2. Cancel Appointment
3. Mark Appointment Completed
Enter choice: 3
Appointment ID: 3002
Appointment marked as Completed.
```

Screenshot 20: 6. Book / Manage Appointment (3)

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable

======================================
     HOSPITAL APPOINTMENT SYSTEM
======================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 7

========== HOSPITAL REPORT ==========
Total Patients     : 6
Total Doctors      : 4
Total Appointments : 5
  Booked           : 3
  Completed        : 1
  Cancelled        : 1
Revenue (completed): Rs.800
-------- Appointments per Doctor --------
Dr. Kiran Mehta (General Physician) : 0
Dr. Anita Joshi (Dermatologist) : 4
Dr. Suresh Rao (Orthopedic) : 1
shyam (MBBS) : 0
======================================
```

Screenshot 21: 7. Report

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable

=====================================
     HOSPITAL APPOINTMENT SYSTEM
=====================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 8

1. Save data now
2. Load data from files
Enter choice: 1
Data saved to files successfully.
```

Screenshot 22: 8. Save / Load (1)

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable

=====================================
     HOSPITAL APPOINTMENT SYSTEM
=====================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 8

1. Save data now
2. Load data from files
Enter choice: 2
Data loaded: 6 patients, 4 doctors, 5 appointments.
```

Screenshot 23: 8. Save / Load (2)

```
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\source> cd ../Executable

=====================================
     HOSPITAL APPOINTMENT SYSTEM
=====================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Book / Manage Appointment (Main Process)
7. Report
8. Save / Load
9. Exit
Enter your choice (1-9): 9
Data saved to files successfully.
Thank you for using the Hospital Appointment System.
Memory released successfully (delete[] used).
PS C:\Users\Divyang\Downloads\IU2541230411_CPP02_Project (1)\IU2541230411_PROJECT\Executable> cd ../Executable
>> .\Project.exe
Data loaded: 6 patients, 4 doctors, 5 appointments.
```

Screenshot 24: 9. Exit

12. Testing

All tests were run on the compiled program. The actual output is copied from the program output.

No.	Test case	Input	Expected output	Actual output
1	Add valid patient	Rahul Sharma, 34, 9876543210, Fever	Patient saved with ID 1001	Patient added successfully. Patient ID = 1001
2	Add valid doctor	Dr. Kiran Mehta, 45, 9811100011, General Physician, 500	Doctor saved with ID 2001	Doctor added successfully. Doctor ID = 2001
3	Invalid menu choice	12, then abc	Error message, ask again	Invalid input! Enter a number from 1 to 9.
4	Invalid name	R@hul	Name rejected	Name must contain only letters, spaces and '.'
5	Invalid phone	12345	Phone rejected	Phone number must be exactly 10 digits.
6	Invalid date	31-02-2026	Date rejected	Invalid date! Use DD-MM-YYYY (example: 15-10-2026).
7	Search patient by ID	1001	Patient record shown	[Patient] ID: 1001 | Name: Rahul Sharma | ...
8	Search missing patient	7777	Error message	Error: Patient not found.
9	Update patient and doctor fee	Patient 1002 new phone/disease; doctor 2001 fee 600	Records updated	Patient updated successfully. Doctor fee updated successfully.
10	Book appointment	Patient 1001, Doctor 2001, 14-10-2026, slot 1	Appointment booked with fee	Appointment booked! Appointment ID = 3001 | Fee to pay: Rs.600
11	Double booking	Patient 1003, Doctor 2001, 14-10-2026, slot 1	Booking refused	Error: This doctor is already booked at that date and time.
12	Mark appointment completed	Appointment 3001	Status becomes Completed	Appointment marked as Completed.
13	Cancel appointment	Appointment 3004	Status becomes Cancelled	Appointment cancelled.
14	Cancel an already cancelled appointment	Appointment 3004	Error message	Error: Only a Booked appointment can be changed.

No.	Test case	Input	Expected output	Actual output
15	Reschedule appointment	Appointment 3003, 17-10-2026, slot 2	New date and time saved	Appointment rescheduled successfully.
16	Delete patient with no appointment	Patient 1005	Patient removed	Patient deleted successfully.
17	Delete patient with booked appointment	Patient 1001	Deletion refused	Error: Patient has a booked appointment. Cancel it first.
18	Report	Option 7	Correct totals and revenue	4 patients, 3 doctors, 4 appointments, Revenue Rs.600
19	Save and Load	Option 8 -> 1, then 8 -> 2	Save and load messages	Data saved to files successfully. Data loaded: 4 patients, 3 doctors, 4 appointments.
20	Persistence after restart	Close and reopen program	Old records loaded	Data loaded: 5 patients, 3 doctors, 5 appointments.

13. Challenges and Solutions

Challenge	Solution
The program crashed or went into an endless loop when a letter was typed instead of a number.	Input is read with getline() as text and converted with stoi() inside try/catch. The question is repeated until the value is valid.
new Patient[capacity] needs objects to be created without values.	A default constructor was added to Person, Patient, Doctor and Appointment.
Hospital needs FileManager and FileManager needs Hospital (circular include).	A forward declaration (class Hospital;) was used in FileManager.h and the full header is included only in FileManager.cpp.
FileManager needs to read the private arrays of Hospital.	FileManager was declared as a friend class of Hospital, so no extra getter functions were required.
The same doctor could be booked twice at the same time.	Appointment::clashesWith() compares doctor, date and slot with every booked appointment before a new one is saved.
Deleting a record from the middle of an array leaves a gap.	All following records are shifted one place to the left and the counter is reduced.
The '|' character typed by the user would break the file format.	readText() does not accept '|'. Damaged lines found while loading are skipped with a message.
IDs could repeat after loading old data.	After loading, the next ID counters are set to the highest ID found plus one.

14. Conclusion

The Hospital Appointment System is a complete menu driven C++ project that covers all requirements of the blueprint: six classes, inheritance, virtual functions and runtime polymorphism, default and parameterized constructors, destructors, dynamic memory with new[] and delete[], file handling with automatic loading and saving, input validation, exception handling and modular .h/.cpp files. The system can add, display, search, update and delete records, book appointments without double booking and create a report.

While building this project I learned how to divide a problem into classes, how a base class and derived classes share code, how virtual functions choose the correct function at run time, why every new[] must have a matching delete[], and how data can be stored and restored using text files. I also learned to test the program with wrong inputs and to draw UML diagrams before and after coding. In future, the project can be extended with a database, a graphical interface, patient history and doctor-wise daily schedules.